

Notas Práticas e Exercícios

Disciplina: Programação Orientada a Objetos

{*Anotações para uso pessoal}

1 Fundamentos

1.1 Hello World e a função main

Para rodar no terminal:

- compilar: `javac HelloWorldApp.java`
- rodar: `java HelloWorldApp`

```
class HelloWorldApp {  
    public static void main ( String [] args ) {  
        System .out . println ( " Hello World !" ) ;  
    }  
}
```

1
2
3
4
5

Em Java, a função main é o ponto de entrada de qualquer programa. Ela é o primeiro método a ser executado quando um programa Java é iniciado. A função main deve ser definida em uma classe e deve ter o seguinte cabeçalho:

```
public static void main ( String [] args )
```

1

A leitura de uma string pode ser feita utilizando a classe Scanner ou a classe BufferedReader. Aqui está um exemplo de como ler uma string utilizando a classe Scanner:

```

import java . util . Scanner ;
public class Exemplo {
    public static void main ( String [] args ) {
        Scanner scanner = new Scanner ( System .in) ;
        System .out . print ( " Digite uma string : " ) ;
        String str = scanner . nextLine ( ) ;
        System .out . println ( "A string digitada foi : " + str ) ;
        scanner . close ( ) ;
    }
}

```

1
2
3
4
5
6
7
8
9
10

Uma explicaç~ao linha por linha do que est´a acontecendo:

- A primeira linha importa a classe Scanner do pacote java.util.
- A segunda linha comea a definiç~ao da classe Exemplo, que ´e o nome do

arquivo. 1

- A terceira linha define o m´etodo main que ´e o ponto de entrada do programa. E aqui que a execuç~ao ´ do programa comea.
- Na quarta linha, criamos um objeto Scanner chamado scanner e o conectamos `a entrada padr~ao do sistema (System.in). Isso nos permitir´a ler a entrada do usu´ario atrav´es do console.
- A quinta linha exibe uma mensagem no console solicitando ao usu´ario que digite uma string.
- A sexta linha lª a string digitada pelo usu´ario e a armazena na vari´avel str.
- A s´etima linha exibe a string digitada pelo usu´ario no console, juntamente com a mensagem "A string digitada foi: ".
- A oitava linha fecha o objeto Scanner.

1.2 Multiplicaç~ao de Matrizes

A multiplicaç~ao de matrizes ´e uma operaç~ao matem´atica que produz uma nova matriz a partir da combinaç~ao das linhas e colunas de duas matrizes pr´e-existentes. Para multiplicar duas matrizes A e B, ´e necess´ario que o n´umero de colunas da matriz A seja igual ao n´umero de linhas da matriz B. O resultado da multiplicaç~ao ´e uma matriz C, onde cada elemento $C(i,j)$ ´e obtido a partir do produto

interno da linha i da matriz A pela coluna j da matriz B.

```
1 import java . util . Scanner ;
2
3 public class MatrizMultiplicacao {
4 public static void main ( String [] args ) {
5 Scanner scanner = new Scanner ( System .in) ;
6
7 // Leitura do tamanho das matrizes
8 System .out . println ( " Digite o n'ú mero de linhas da primeira matriz :") ; 9 int linhas1 =
scanner . nextInt () ;
10 System .out . println ( " Digite o n'ú mero de colunas da primeira matriz :") ;
11 int colunas1 = scanner . nextInt () ;
12 System .out . println ( " Digite o n'ú mero de linhas da segunda matriz :") ; 13 int linhas2
= scanner . nextInt () ;
14 System .out . println ( " Digite o n'ú mero de colunas da segunda matriz :") ; 15 int
colunas2 = scanner . nextInt () ;
16
17 // Verifica se as matrizes s'ao compat 'i veis para multiplica c,~ao 18 if (
colunas1 != linhas2 ) {
19 System .out . println ( "As matrizes n'ao s'ao compat 'i veis para multiplica
c,~ao." ) ;
20 return ;
21 }
22
23 // Leitura dos elementos da primeira matriz
24 System .out . println ( " Digite os elementos da primeira matriz :") ; 25 int [][]
matriz1 = new int [ linhas1 ][ colunas1 ] ;
26 for ( int i = 0 ; i < linhas1 ; i++ ) {
27 for ( int j = 0 ; j < colunas1 ; j++ ) {
28 matriz1 [ i ][ j ] = scanner . nextInt () ;
29 }
30 }
31
32 // Leitura dos elementos da segunda matriz
33 System .out . println ( " Digite os elementos da segunda matriz :") ; 34 int [][]
matriz2 = new int [ linhas2 ][ colunas2 ] ;
35 for ( int i = 0 ; i < linhas2 ; i++ ) {
36 for ( int j = 0 ; j < colunas2 ; j++ ) {
37 matriz2 [ i ][ j ] = scanner . nextInt () ;
38 }
39 }
40
41 // Multiplica c,~ao das matrizes
42 int [][] resultado = new int [ linhas1 ][ colunas2 ] ;
43 for ( int i = 0 ; i < linhas1 ; i++ ) {
44 for ( int j = 0 ; j < colunas2 ; j++ ) {
45 for ( int k = 0 ; k < colunas1 ; k++ ) {
46 resultado [ i ][ j ] += matriz1 [ i ][ k ] * matriz2 [ k ][ j ] ; 47 }
48 }
49 }
```

```

50
51 // Exibi c,~ao da matriz resultado
52 System .out . println ("O resultado da multiplica c,~ao 'e:") ; 53 for (int
i = 0; i < linhas1 ; i++) {
54 for (int j = 0; j < colunas2 ; j++) {
55 System .out . print ( resultado [i][j] + " ") ;
56 }
57 System .out . println () ;
58 }
59 }
60 }

```

1.3 Atividade instanceof

A hierarquia das classes Java segue a seguinte ordem:

- Object: 'e a classe raiz da hierarquia de classes Java e todas as outras classes herdam dela.
- Number: 'e a classe pai das classes que representam valores num'ericos, como Integer, Long e Double.
- Integer: representa um valor inteiro.
- Long: representa um valor inteiro maior do que o Integer.
- Double: representa um valor decimal de ponto flutuante.
- Boolean: representa um valor booleano (verdadeiro ou falso).

Vale ressaltar que existem muitas outras classes na hierarquia de classes Java que n~ao foram mencionadas nesta lista, incluindo classes para lidar com strings, datas, cole,~oes, entre outras. Obs. usar Integer.valueOf(123); para instanciar.

```

public class Exemplo {

    public static void main ( String [] args ) {
        Integer intObj = new Integer (123) ;
        Integer intObj1 = Integer . valueOf (123) ;
        Long longObj = new Long (1234567890) ;
        Double doubleObj = new Double (12.345) ;
        Boolean boolObj = new Boolean ( true ) ;
        Object [] objArray = { intObj , longObj , doubleObj , boolObj } ;
        for (int i = 0; i < objArray . length ; i++) {
            if ( objArray [i] instanceof Number ) {
                System .out . println ( objArray [i]. toString () + " 'e um
                    Number ." ) ;
            } else {
                System .out . println ( objArray [i]. toString () + " N~AO 'e um
                    Number ." ) ;
            }
        }
    }
}

```

1
2
3
4
5
6
7
8
9
10
11
12

13
14

15
16
17
18

1.4 Palindromo

Um palíndromo é uma palavra, frase, número ou qualquer sequência de caracteres que pode ser lida da mesma maneira da esquerda para a direita ou da direita para a esquerda. Por exemplo, 'arara'. No código abaixo, a entrada do usuário é lida como uma string e os espaços em branco são removidos e as letras são convertidas para minúsculas. Em seguida, a string original é invertida e comparada com a string original para verificar se é um palíndromo. A saída do programa informa se a string é um palíndromo ou não.

```

1 import java . util . Scanner ; // Importa a classe Scanner para ler a entrada do usu ´ario
2
3 public class Palindromo {
4 public static void main ( String [] args ) {
5 // Cria um objeto Scanner para ler a entrada do usu ´ario 6 Scanner sc = new
Scanner ( System .in) ;
7
8 // Pedes para o usu ´ario digitar uma string
9 System .out . print ( " Digite uma string : " ) ;
10 String str = sc. nextLine ( ) ;
11
12 // Remove os espa c,os em branco da string e converte para letras min ´u sculas
13 str = str . replaceAll ( "\\s", "" ) . toLowerCase ( ) ;
14
15 // Cria uma string vazia para armazenar a string invertida 16 String invertida =
"" ;
17
18 // Inverte a string original
19 for ( int i = str . length ( ) - 1; i >= 0; i - - ) {

20 invertida += str . charAt ( i ) ;
21 }
22
23 // Verifica se a string original ´e igual `a string invertida 24 if ( str . equals
( invertida ) ) {
25 System .out . println ( "A string ´e um pal ´ı ndromo ." ) ; 26 } else {
27 System .out . println ( "A string nˆao ´e um pal ´ı ndromo ." ) ; 28 }
29 }
30 }

```

4

1.5 Criando uma classe Retangulo

Uma classe ´e uma estrutura/modelo b´asico para definir objetos e seus comportamentos. Uma classe ´e composta por campos (vari´aveis) e m´etodos (fun,cˆoes), que podem ser acessados e usados para criar instˆancias da classe. As instˆancias de uma classe em Java sˆao objetos criados a partir dessa classe. Cada instˆancia representa uma 'cˆopia' da classe, com seus pr´oprios valores para os campos da classe. Vocˆe pode criar v´arias instˆancias de uma mesma classe, cada uma com seus pr´oprios valores para os campos. Por exemplo, se vocˆe tiver uma classe Pessoa com campos para nome e idade, vocˆe pode criar v´arias instˆancias dessa classe para representar diferentes pessoas, cada uma com seu pr´oprio nome e idade. Para criar uma instˆancia de uma classe em Java, vocˆe precisa usar a palavra-chave **new**, seguida pelo nome da classe e parˆenteses (com ou sem argumentos dependendo do construtor).

O cˆodigo abaixo define uma classe Retangulo (que estar´a em um arquivo Retangulo.java) em Java que tem duas vari´aveis de instˆancia privadas (largura e altura) e m´etodos p´ublicos para calcular a ´area e o per´ımetro do retˆangulo, bem como m´etodos get e set para acessar e modificar as vari´aveis de instˆancia.

O construtor da classe Retangulo recebe dois argumentos (largura e altura) e usa o operador **this** para atribuir esses valores `as vari´aveis de instˆancia correspondentes. O m´etodo calcularArea() simplesmente retorna o produto da largura e altura do retˆangulo. O m´etodo calcularPerimetro() retorna o per´ımetro do retˆangulo, que ´e o dobro da soma da largura e altura.

Os m´etodos get e set permitem que outros objetos acessem e modifiquem as vari´aveis de

instância privadas largura e altura. O método `getLargura()` retorna o valor atual da largura e o método `setLargura()` permite que a largura seja modificada. O mesmo vale para o método `getAltura()` e `setAltura()`.

```
1 public class Retangulo {
2     private double largura ;
3     private double altura ;
4
5     public Retangulo ( double largura , double altura ) {
6         this . largura = largura ;
7         this . altura = altura ;
8     }
9
10    public double calcularArea () {
11        return largura * altura ;
12    }
13
14    public double calcularPerimetro () {
15        return 2 * ( largura + altura ) ;
16    }
17
```

5

```
18    public double getLargura () {
19        return largura ;
20    }
21
22    public void setLargura ( double largura ) {
23        this . largura = largura ;
24    }
25
26    public double getAltura () {
27        return altura ;
28    }
29
30    public void setAltura ( double altura ) {
31        this . altura = altura ;
32    }
33 }
```

Agora você pode criar objetos do tipo `Retangulo` e usar os métodos do objeto para calcular a área e o perímetro. Também é possível acessar e modificar os atributos do objeto usando os métodos `get` e `set`.

```

public class Exemplo {

    public static void main ( String [] args ) {
        Retangulo retangulo = new Retangulo (10 , 5) ;
        double area = retangulo . calcularArea () ;
        double perimetro = retangulo . calcularPerimetro () ;
        double largura = retangulo . getLargura () ;
        retangulo . setLargura (20) ;

    }

}

```

1
2
3
4
5
6
7
8
9
10
11
12

2 Exercícios Práticos

1. Escreva um programa que leia três inteiros e imprima a soma, a média, o produto e o maior e o menor elemento lido.
2. Escreva um programa que leia o raio de um círculo e imprima o diâmetro, a área e a circunferência do círculo. Os resultados devem ser escritos com duas casas decimais.
3. Crie um programa que leia dois números inteiros e determine se um é múltiplo do outro.
- 4.

Escreva um programa que lê um caractere e retorna seu código na tabela ASCII (um número inteiro).

5. Escreva um programa que lê 15 números double e através de uma função determine quais são os dois maiores deles, que não devem ser repetidos.
6. Uma grande companhia paga seus vendedores com salários fixos mais comissões sobre as vendas. Um vendedor recebe R\$200,00 mais 9% de comissão sobre todas as suas vendas. Escreva um programa que lê o valor total das vendas de um vendedor e calcule seu salário. Seu programa deve computar o salário de diversos vendedores, e parar quando o valor informado for -1.

7. Escreva um programa que estima o valor da constante e pela fórmula (o usuário deve informar o número de termos deste somatório, que define a precisão do resultado):

$$e = (1/1!) + (1/2!) + (1/3!) + \dots (1)$$

8. Crie um programa que leia 12 números inteiros em qualquer ordem e imprima uma listagem contendo o número lido e uma mensagem “PAR” ou “IMPAR” conforme o número.
9. Crie um programa que armazene 12 números em um array e determine qual a porcentagem de números menores que 8 e qual a porcentagem de números maiores que 10.
10. Quadrado Mágico. Um Quadrado Mágico é uma tabela quadrada de lado N , onde a soma dos números das linhas, das colunas e das diagonais principal e secundária é constante, sendo que nenhum destes números se repete. Crie um programa para verificar (não confunda com resolver) se uma matriz realmente é um quadrado mágico. Para isso primeiro o usuário entra com o tamanho da matriz e depois digita os valores. Um exemplo de quadrado mágico:

8	1	6
3	5	7
4	9	2

11. Escreva um programa que leia uma string do usuário e verifique se ela contém apenas caracteres únicos.
12. Após implementar e estudar o exemplo da Seção 1.5, faça:
- Escreva uma classe Quadrado que tenha uma variável de instância para representar o lado do quadrado. Adicione métodos para calcular a área e o perímetro do quadrado.
 - Escreva uma classe Círculo que tenha uma variável de instância para representar o raio do círculo. Adicione métodos para calcular a área e a circunferência do círculo.
 - Escreva uma classe Triângulo que tenha três variáveis de instância para representar os lados do triângulo. Adicione métodos para calcular a área e o perímetro do triângulo.
 - Escreva um programa que crie objetos do tipo Quadrado, Círculo e Triângulo e imprima suas áreas e perímetros.
13. Crie uma classe Conta que tenha os seguintes campos privados: Nome; CPF (pode usar long); Saldo R\$; Número da conta; Agência. Para cada um dos campos disponibilize um método set e get. Faça um método para depósito (que recebe um valor) e outro método para saque da conta (que recebe um valor e retorna ele).

